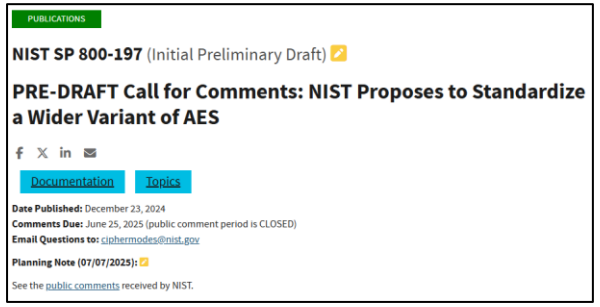
Adapting AES-Oriented Optimizations to Rijndael-256: Cortex-M4, ARMv8-A, and CUDA

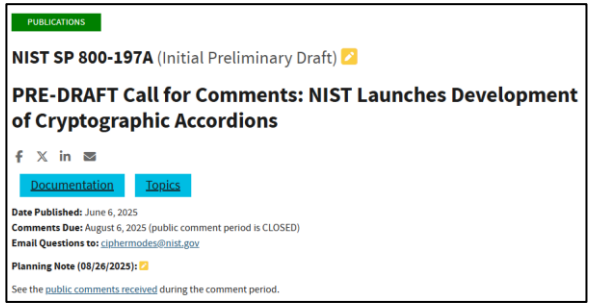
Siwoo Eum, Minho Song, Minjoo Sim, Anupam Chattopadhyay, Hwajeong Seo
(Hansung University / Nanyang Technological University)

1. Intro: What is the Rijndael-256

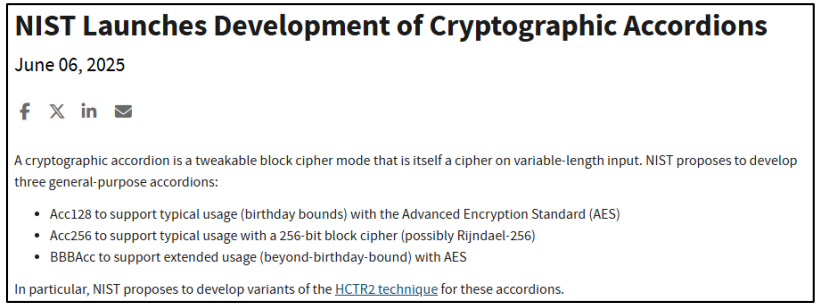
- Wider Variant of AES (SP 800-197)
  - Resolves the birthday-bound issue with a fixed large block size
- Variable-length input adds flexibility(Padding issue)
  - tweak values resolve key reuse
- NIST plans three general-purpose accordions block cipher mode
  - Acc128, Acc256, BBBAcc



SP 800-197



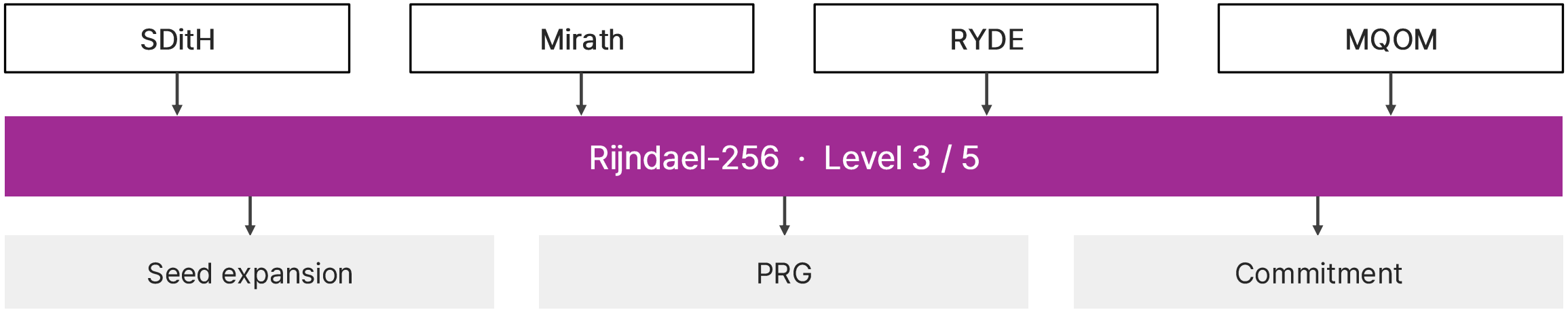
SP 800-197A



Accordions Mode

1. Intro: Why Rijndael-256

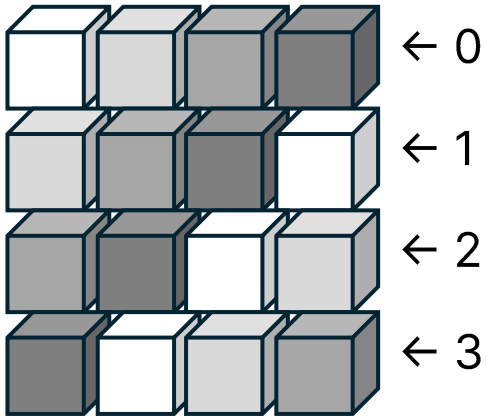
- PQC signatures call Rijndael-256 as a repeated core operation
  - Seed expansion, PRG and commitments invoke it many times per signature
- Its speed directly shapes signing and verification cost
- This paper evaluates the Level 3 / 5 parameter sets



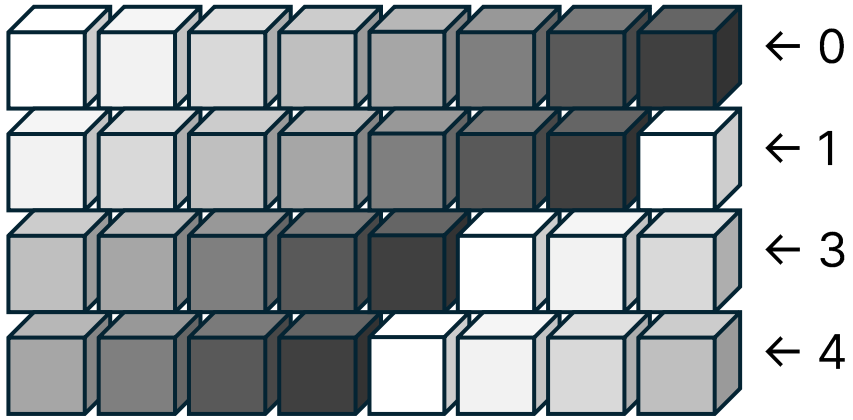
2. Background: What is the Rijndael-256

- AES uses a 128-bit block, Rijndael-256 uses a 256-bit block
  - Don't confuse it with AES-256, where 256 is the key size.

- AES and Rijndael-256 differ in State size(block size) and ShiftRows offsets



AES

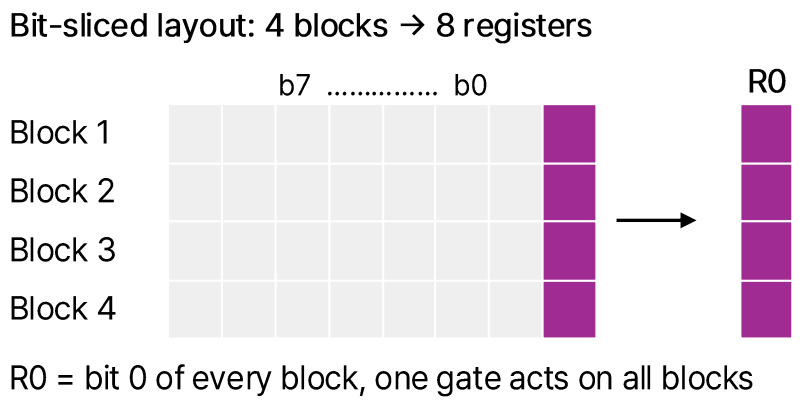


Rijndael-256

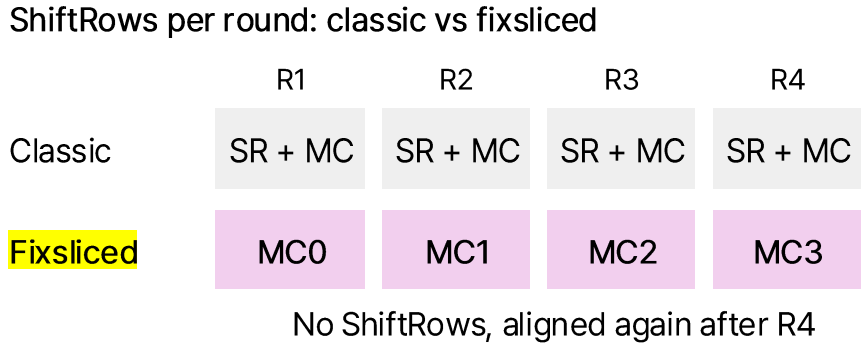
2. Background: Implementation Method of AES (1/2)

- T-table: S-box, ShiftRows and MixColumns merged into 4 lookup tables
  - Four 1 KB tables (256 × 32-bit), one round = 16 lookups + XORs
  - Fast in software, but secret-indexed lookups leak through cache timing

- Bitslicing: S-box as logic gates, not tables
  - One register per bit position, processing many blocks at once
  - No table lookups, so no cache-timing leaks (constant time)
  - ShiftRows and MixColumns become bit moves and XORs

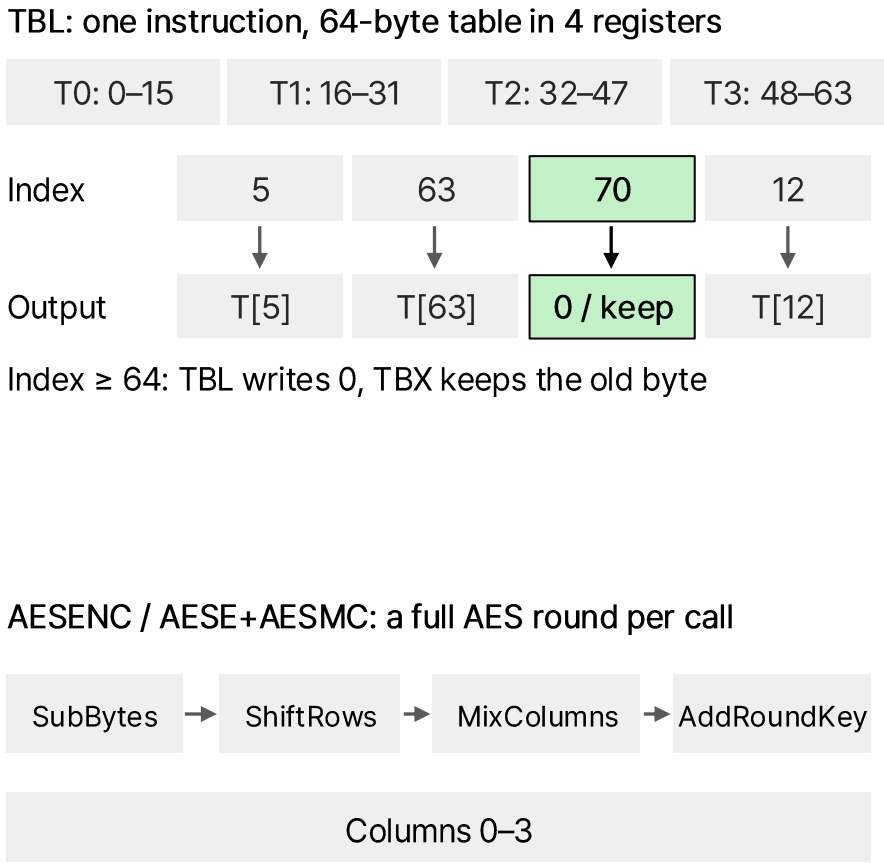


- Fixslicing: skips most ShiftRows
  - Slices stay in place, MixColumns variants absorb the skipped shifts
  - AES's 4-column state realigns every 4 rounds
  - Fastest constant-time AES on 32-bit MCUs (Adomnicai & Peyrin, 2021)



2. Background: Implementation Method of AES (2/2)

- TBL / TBX: NEON byte table lookup
  - Picks bytes from up to 4 vector registers (64 B) in one instruction
  - Out-of-range index: TBL writes 0, TBX keeps the old byte
  - Handles ShiftRows permutations and in-register S-box lookups
- ARM Cryptographic Extension: one AES round per instruction
  - ARMv8 AESE + AESMC,
    AESENC / AESENCLAST in AES-NI x86
  - Fixed to a 128-bit state and AES ShiftRows offsets
  - Rijndael-256 must reshuffle bytes around each call



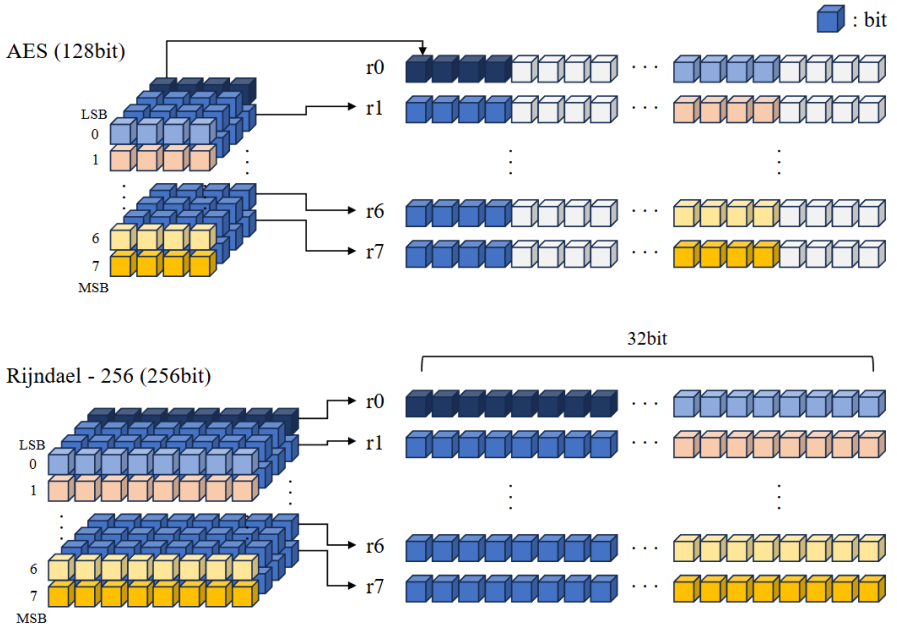
3. Overview: Each platform needs its own adaptation strategy

- Why don't AES optimizations carry over to Rijndael-256 as is?

Platform	Strategy	Key technique	Result
Cortex-M4	Wider classical bitslicing	• Drop fixslicing • explicit ShiftRows permutation	4,962 cyc/block
AArch64	Reuse with reshuffling	• TBL pre-shuffle + AESE • folded round keys	6,520 MB/s Apple M2
CUDA	Amortize the mismatch	• Shared-memory T-tables • Bank conflicts removed	81.16 GiB/s RTX 4080

4. Cortex-M4: One Rijndael-256 block fills the registers

- No AES instructions or SIMD → bitslicing for CT
  - State split into 8 bit planes
- Problem: one bit plane = one full 32-bit register
  - One block occupies r4–r11, only 6 of 14 usable registers spare
  - No multi-block parallelism, S-box spills and reloads via the stack
- Reuse: AES building blocks carry over unchanged
  - S-box circuit, SWAPMOVE packing, MixColumns logic



4. Cortex-M4: Fixslicing costs 5× more on Rijndael-256

- Fixslicing skips ShiftRows, MixColumns absorbs the offsets
  - MixColumns XORs adjacent rows, so skipped shifts leave row-to-row offsets
- AES: one 8-bit ROR fixes all rows, Rijndael-256: 8 / 16 / 8 / 32-bit rotations
  - No single ROR, each variant unwinds and rewinds byte by byte
- Decision: drop fixslicing → wider classical bitslicing
  - ShiftRows handled as an explicit permutation

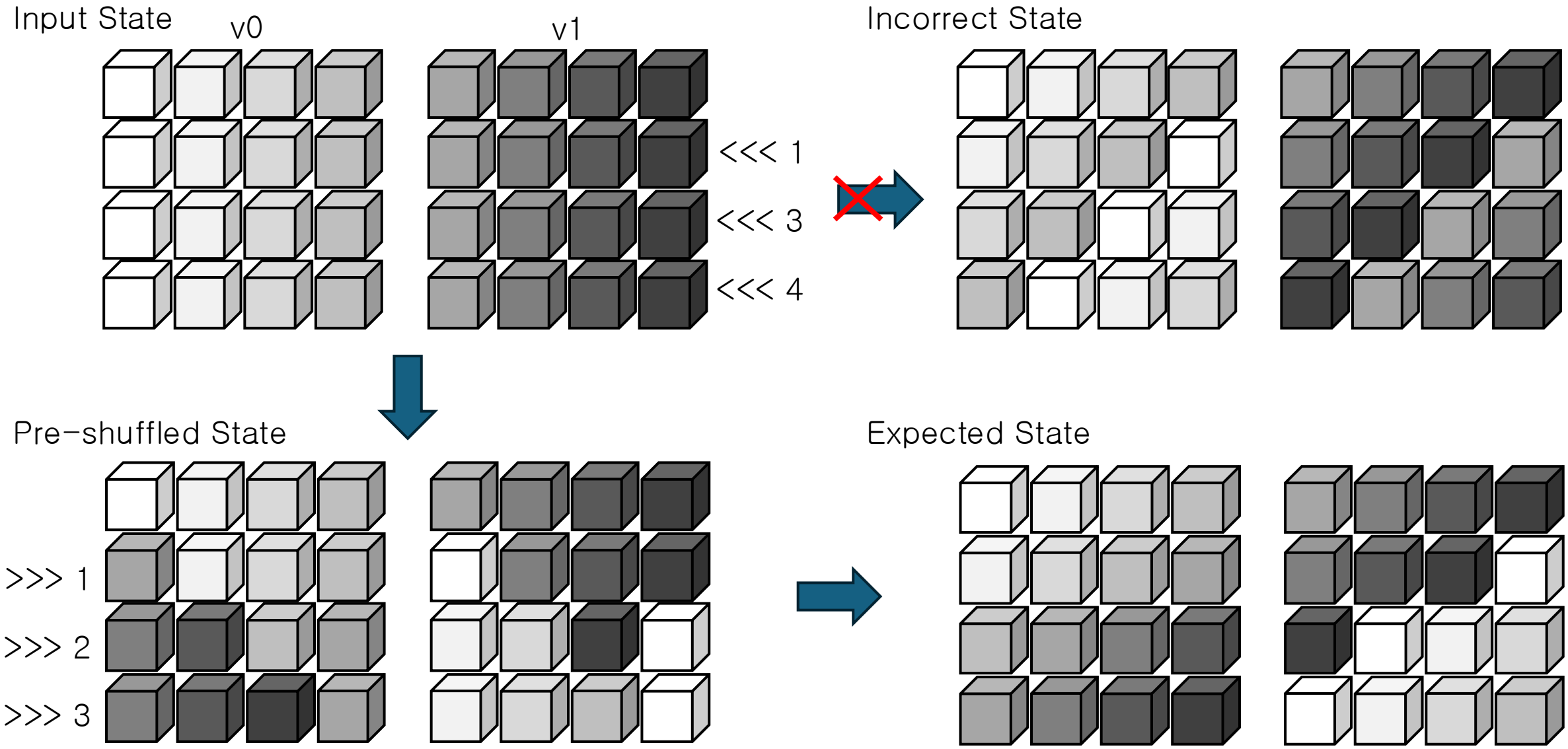
	AES fixslicing	Rijndael-256 fixslicing
Inter-row offsets	(1, 1, 1, 1)	(1, 2, 1, 4)
Rotation fix	One ROR by 8	Byte-by-byte unwind / rewind
Instructions per variant	~40	176 avg, 227 max (>5×)
Realignment period · variants	4 rounds · 2–4	8 rounds · 8

4. Cortex-M4: Bitslicing is the fastest constant-time option

- vs. Fixslice: faster and smaller on every metric
  - 10% faster encryption, 1.7× faster key schedule, 2.7× smaller ROM
- 56% slower than T-table ASM: cost of CT + loss of 2-block parallelism

Implementation	Encrypt (cyc)	Key sched. (cyc)	CT
Reference C	12,374	3,525	No
T-table C	3,785	3,525	No
T-table ASM	3,174	3,525	No
Bitslice ASM	**4,962**	**5,931**	**Yes**
Fixslice ASM	5,481	10,299	Yes

5. AArch64: A pre-shuffle lets AESE run unchanged



5. AArch64: Folding the round key into AESE

- Problem: natural-order key hits the wrong columns
  - AESE XORs the key while the state is in shuffled order
- EOR variant: zero key into AESE, EOR K_i after AESMC
  - Key schedule unchanged · 2,193.9 MB/s (1 block)
- Folded variant: $K_i^{pre} = TBL(K_i)$, once per key
  - 4,196.7 MB/s → 6,520.0 MB/s with 4 blocks

Non-final folded round · 6 instructions

```
// pre-shuffle
TBL  v2, {v0, v1}, v16
TBL  v3, {v0, v1}, v17

// ARK(Kpre) + SB + SR(0,1,2,3)
AESE  v2, v20
AESE  v3, v21

// MixColumns
AESMC v0, v2
AESMC v1, v3
```

5. AArch64: Slowdown vs. equal-round AES-256 is 1.09×–2.40×

- Why x86 is smaller: one VPERMB amortized over 4 blocks
  - One 512-bit VAESENC covers 4 AES lanes (2 R256 blocks) with a single VPERMB
  - ARM's 128-bit AESE covers half an R256 block, needing 2 TBLs per block per round.
  - x86 reproduces Drucker–Gueron
    - https://research.ibm.com/publications/optimized-implementation-of-rijndael256-on-modern-x86-64-platforms

Best interleave	Apple M2 (MB/s ↑)	Cortex-A76 (MB/s ↑)	x86-64 (cpb ↓)
AES-128	15,252.6	3,441.0	0.188
AES-256	11,721.6	2,435.6	0.234
Rijndael-256	6,520.0	1,014.1	0.255
vs. AES-256	1.80×	2.40×	1.09×

5. AArch64: NEON 4PT is the CT fallback, 12% ahead of IL 4PT

NEON 4PT

One block per register pair

v0	A·c0	A·c1	A·c2	A·c3
v1	A·c4	A·c5	A·c6	A·c7
v2	B·c0	B·c1	B·c2	B·c3
v3	B·c4	B·c5	B·c6	B·c7
v4	C·c0	C·c1	C·c2	C·c3
v5	C·c4	C·c5	C·c6	C·c7
v6	D·c0	D·c1	D·c2	D·c3
v7	D·c4	D·c5	D·c6	D·c7

ShiftRows: 2 TBLs per block, one 2-register table each

NEON IL 4PT

One column of all 4 blocks

v0	A·c0	B·c0	C·c0	D·c0
v1	A·c1	B·c1	C·c1	D·c1
v2	A·c2	B·c2	C·c2	D·c2
v3	A·c3	B·c3	C·c3	D·c3
v4	A·c4	B·c4	C·c4	D·c4
v5	A·c5	B·c5	C·c5	D·c5
v6	A·c6	B·c6	C·c6	D·c6
v7	A·c7	B·c7	C·c7	D·c7

ShiftRows: rows come from different registers, a serial MOV → BIT×3 → MOV chain

Instr. per round	4PT	IL 4PT
SubBytes	59	59
ShiftRows	16	41
MixColumns	80	80
AddRoundKey	9	17
Round total	164	197 (+20%)
Layout conversion	0	~50
Measured (MB/s)	480 ↑	429 ↓

- Without AES instructions, NEON keeps the S-box in 16 registers (4 TBL/TBX, no lookups)
  - Cortex-A72: T-table 54 MB/s vs NEON 26 MB/s
- The 12% gap comes only from ShiftRows, AddRoundKey and layout conversion
  - SubBytes and MixColumns cost the same in both, 4PT reaches 480 MB/s on M2 (4.0× single-block)

6. CUDA: The bottleneck was bank conflicts, not occupancy

- ✓ Compact: 8 KiB, 86.9% occupancy
  - Eight T-tables as is, accepts conflicts
  - High occupancy hides the latency
- ✓ Conflict-free (CF): 36 KiB, ~16%
  - Te0 replicated to 256×32, a column per thread
  - Final-round T4 (4 KiB) is not replicated

```
// naive: one column at once → 65.5M conflicts
Te0_s[i][b] = val;
// staggered: offset by lane → 32 banks
Te0_s[i][(b + lane) & 31] = val;
```

Metric	Compact	CF naive	CF staggered
Occupancy	86.9%	16.2%	15.6%
Bank conflicts	37,303	65,533,970	389,098
IPC	59.5	52.5	87.9
ECB (GiB/s)	79.93	52.84	81.16

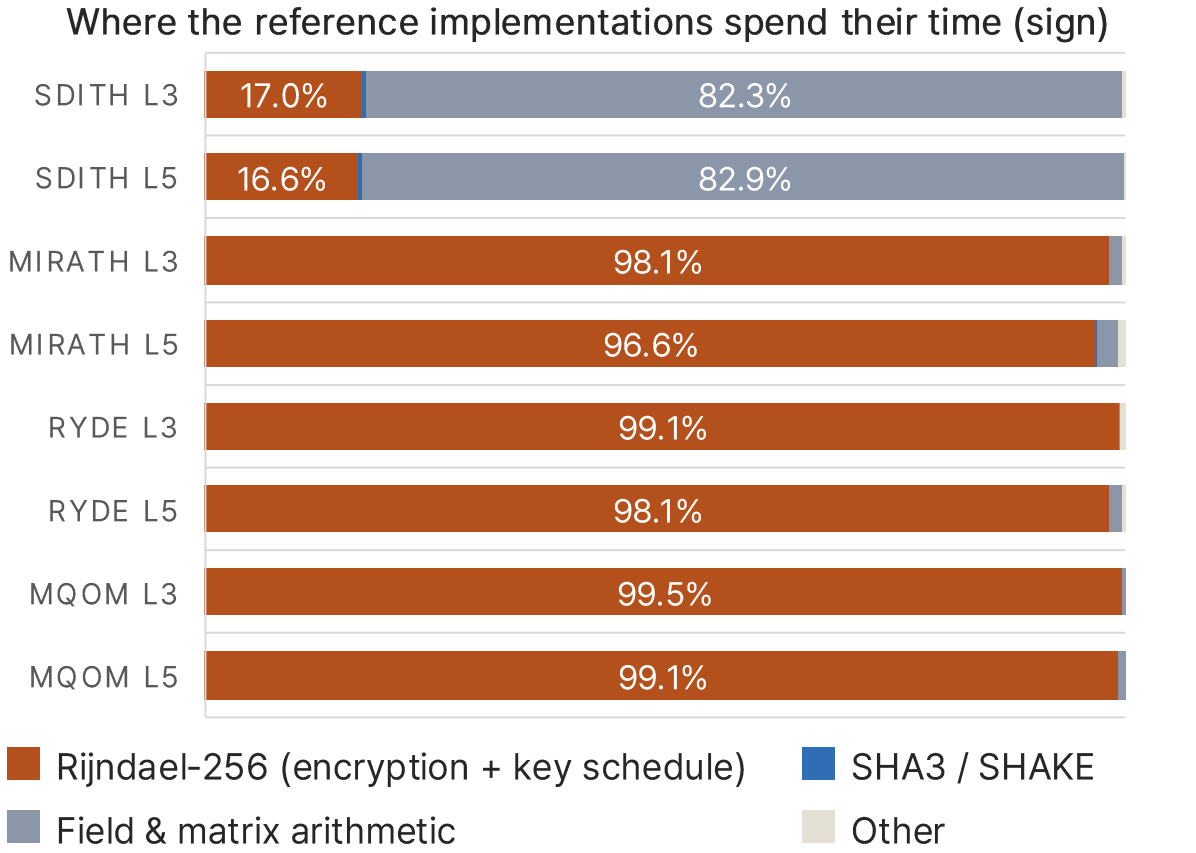
- Staggering writes by lane: 99.4% fewer conflicts, SM cycles 1.59× better (0.97× of Compact)
- 1/5.5 the occupancy, yet IPC 52.5 → 87.9 with 43% more instructions

7. PQC: R256's share of runtime sets the speedup

Speedup of ARM Crypto over Reference, Apple M2

Scheme	R256 %	Sign L3	Sign L5	Verify L3	Verify L5
SDitH	16–17%	1.18×	1.18×	1.20×	1.19×
Mirath	96.8–99.3%	51.4×	30.5×	133.2×	85.5×
RYDE	98.3–99.3%	106.2×	56.6×	121.9×	67.2×
MQOM	99.1–99.6%	114.3×	85.0×	155.6×	125.5×

- Mirath, RYDE, MQOM: >96% in R256
  - Sign 30–114×, verify up to 155.6× faster
- SDitH: 82% on GF(2^{192}) products



Apple M2 · reference code · 1 ms stack sampling

8. Conclusion: R256 needs per-platform redesign

- Not a widened AES: AES optimizations break on R256
  - Wider 4×8 state and ShiftRows offsets (0,1,3,4)
- Analysis: why each optimization breaks, per platform
  - Cortex-M4, AArch64 and CUDA each fail differently
- Implementation: dedicated code for all three platforms
  - Cross-platform evaluation, including CT properties
- Integration: 4 NIST PQC signatures, up to 155.6× faster
  - Gains track R256's share of runtime

Thank you

Questions

Siwoo Eum · shuraatum@gmail.com
Hansung University